

What is pandas?

Think of pandas as **Excel in Python**. It helps you work with tabular data (rows & columns) in a flexible, programmatic way.

- It has two main parts:
 - A **Series** – like a single column in a spreadsheet (just one list of values).
 - A **DataFrame** – like the whole spreadsheet: many columns, many rows.
 - It makes it easy to: read data, clean it, slice/filter it, do calculations, and save it again.
-

✅ Why use pandas?

Because raw Python lists/dictionaries are **tough** when working with lots of rows and columns. Pandas gives you tools that are:

- **Structured** (named columns, indexes)
 - **Efficient** – designed for working with big datasets/tables
 - **Rich** – has built-in methods for common tasks (missing values, grouping, filtering)
 - Great for data analysis, not just raw values
-

Basic workflow – with a simple example

Here's how it typically works, broken down to steps you can remember:

1. **Import and load data**
2. import pandas as pd
3. df = pd.read_csv("mydata.csv")

You load your table into a DataFrame (df).

4. **Inspect the data**
5. df.head() # see first few rows
6. df.info() # see data types, missing values
7. df.describe() # summary stats for numeric columns

8. Clean or fix data

- Deal with missing values:
- `df = df.dropna()` # drop rows with missing values
- `df["Age"] = df["Age"].fillna(df["Age"].mean())` # fill with mean
- Rename columns, convert types, remove duplicates.

9. Filter/select or compute new columns

10. `subset = df[df["Department"] == "IT"]` # only rows where Department == IT

11. `df["Bonus"] = df["Salary"] * 0.10` # create new column Bonus

12. Aggregate / group / analyze

13. `avg_salary = df.groupby("Department")["Salary"].mean()`

This gives you, for each department, the average salary.

14. Export/save results

15. `df.to_csv("cleaned_data.csv", index=False)`



Very simple analogy

- Imagine you have a **sheet of paper** with names, ages, salaries.
- pandas = your *smart assistant* that:
 - can **read** that sheet into memory
 - let you **ask questions**: “Show me only people aged > 30”
 - let you **modify** it: “Add a column for bonus”
 - let you **summarise** it: “What’s average salary per department?”
 - and **save it** back to a new sheet.

Key terms to remember

- **DataFrame**: the main “table” object (rows + columns)
- **Series**: a single column (or row) in a DataFrame

- **Index:** the labels for rows (and/or columns)
- **Missing values:** some entries might be empty/NaN — pandas has tools to handle them
- **Groupby / aggregation:** splitting your data into groups, then summarising
- **Filtering / boolean indexing:** selecting rows based on conditions

1. Reading & Writing Data (I/O)

This is how you bring data into pandas and how you export it.

Reading

```
import pandas as pd

# Read a CSV file
df = pd.read_csv("data.csv")

# Read an Excel file
df2 = pd.read_excel("data.xlsx", sheet_name="Sheet1")

# Read a JSON file
df3 = pd.read_json("data.json")
```

As noted in the pandas tutorial: you can load from CSV, Excel, JSON, SQL, etc. For example, `pd.read_csv()` is very commonly used.

Writing

```
# Write the DataFrame to CSV
df.to_csv("output.csv", index=False)

# Write to Excel
df.to_excel("output.xlsx", index=False)

# Write to JSON
df.to_json("output.json")
```

Writing data out lets you save your cleaned/transformed data for sharing or further use.

Why this matters: Before you can manipulate data you often need to read it in. After manipulation you'll often export results.

2. Data Manipulation

This covers picking, dropping, renaming, reshaping — basically doing things to make the data ready.

Examples:

```
# Suppose we have this DataFrame:

data = {
    "Name": ["Alice", "Bob", "Charlie", "David"],
    "Age": [25, 30, 35, 40],
    "Department": ["HR", "IT", "IT", "Finance"],
    "Salary": [50000, 60000, 65000, 70000]
}

df = pd.DataFrame(data)

# Drop a column
df2 = df.drop(columns=["Department"])

# Rename a column
df3 = df.rename(columns={"Salary": "AnnualSalary"})

# Add a new column
df["Bonus"] = df["AnnualSalary"] * 0.10 # assuming you renamed

# Change data type
df["Age"] = df["Age"].astype(float)

# Re-indexing: set Department as index
df4 = df.set_index("Department")
```

From articles: dropping columns, renaming, etc. are key manipulation tasks. Also manipulation is described as “dropping columns/rows, renaming columns, handling duplicates, grouping data” etc.

Why this matters:

Raw data often has many extra columns, wrong names, wrong types, etc. Manipulation helps you **shape** the data into a form you can work with.

3. Data Cleaning

Cleaning means dealing with missing values, duplicates, wrong formats, etc, so your data is reliable.

Examples:

```
# Detect missing values
df.isnull().sum()

# Drop rows with any missing values
df_clean1 = df.dropna()

# Fill missing values (e.g., with mean)
mean_age = df["Age"].mean()
df_clean2 = df.fillna({"Age": mean_age})

# Remove duplicates
df_no_dup = df.drop_duplicates()

# Rename columns or convert strings
df_clean2 = df_clean2.rename(columns={"Name": "EmployeeName"})
```

From Real Python: cleaning often involves dropping columns, cleaning strings, renaming, dealing with duplicates. [_](#)

And broader guides say cleaning is crucial for pre-analysis.

Why this matters:

If you analyse bad data (lots of missing values, inconsistent formats) your results will be unreliable. Clean data → trustworthy analysis.

4. Data Transformation

Transformation means changing the data's shape or values: deriving new columns, normalizing, pivoting, etc.

Examples:

```
# Derive new column
df["SalaryChange"] = df["AnnualSalary"] - df["Bonus"]

# Map values: suppose Department codes need mapping
dept_map = {"HR":"Human Resources", "IT":"Information Technology", "Finance":"FinOps"}
df["DeptFull"] = df["Department"].map(dept_map)

# Pivot / reshape:
pivot_df = df.pivot_table(index="Department", values="AnnualSalary", aggfunc="mean")
```

Transformation is part of “shape, summarise, extract insights” from data.

Why this matters:

Often you need to create new features (columns) for analysis, or reshape data so that the format is suitable for grouping/plotting/modeling.

5. Grouping & Aggregation

Group data by one or more keys, then compute summary statistics (mean, sum, count...) for each group.

Examples:

```
# Using the earlier df
# Average salary by department
avg_salary_by_dept = df.groupby("Department")["AnnualSalary"].mean()
print(avg_salary_by_dept)
```

```
# Count of people by department
count_by_dept = df.groupby("Department")["Name"].count()

# More complex: multiple aggregations
agg_dept = df.groupby("Department").agg({
    "AnnualSalary": ["mean", "max", "min"],
    "Bonus": "sum"
})
```

Guides call this the “split-apply-combine” pattern: you split data into groups, apply functions, then combine results.

Also grouping is listed under manipulation/analysis tasks.

Why this matters:

Most analytical tasks ask “by group” questions — e.g., what’s average salary by department, how many users by type, what’s the sum of transactions by month, etc. Grouping makes these easy.

6. Data Filtering

Filtering means selecting subsets of the data by conditions — e.g., rows that meet certain criteria.

Examples:

```
# Filter rows where salary > 60000
high_salary = df[df["AnnualSalary"] > 60000]

# Multiple conditions
it_dept_high = df[(df["Department"] == "IT") & (df["AnnualSalary"] > 60000)]

# Using .loc
subset = df.loc[df["Age"] < 30, ["Name", "Department", "AnnualSalary"]]
```

Filtering is part of manipulation and is frequently used. For example: “extract all rows where Gender = Female AND Education = Not Graduate”.

Why this matters:

Often you only want to look at a subset of the data: e.g., suspicious login attempts, users older than X, transactions from a certain region. Filtering lets you isolate that subset easily.

7. Data Analysis (Exploration + Summary)

Once the data is cleaned, transformed, grouped, filtered — you can **analyse** it: compute summary statistics, look for patterns, prepare for visualization or modelling.

Examples:

```
# Basic descriptive stats
df["AnnualSalary"].describe()

# Calculate mean, median, mode
mean_salary = df["AnnualSalary"].mean()
median_salary = df["AnnualSalary"].median()
mode_salary = df["AnnualSalary"].mode()

# Correlation
corr = df[["Age","AnnualSalary","Bonus"]].corr()

# Create new insights
df["SalaryPerAge"] = df["AnnualSalary"] / df["Age"]
```

From the tutorial: analysis includes summary stats, modes, medians, creating new columns from existing ones.

Why this matters:

This is the stage where you derive insights: knowing how data is distributed, relationships between columns, whether something stands out.

Putting It All Together: A Mini End-to-End Example

Here's a combined example to illustrate many of the above steps:

```
import pandas as pd

# 1. Read data
df = pd.read_csv("employees.csv")

# 2. Clean data
df = df.dropna(subset=["AnnualSalary", "Age"]) # remove rows missing key fields
df["Department"] = df["Department"].fillna("Unknown")

# 3. Manipulate / transform
df = df.rename(columns={"Salary": "AnnualSalary"})
df["Bonus"] = df["AnnualSalary"] * 0.10
df["SalaryPerAge"] = df["AnnualSalary"] / df["Age"]

# 4. Filter data
young_high_salary = df[(df["Age"] < 30) & (df["AnnualSalary"] > 60000)]

# 5. Group / aggregate
summary_by_dept = df.groupby("Department").agg({
    "AnnualSalary": ["mean", "max", "min"],
    "Bonus": "sum",
    "Name": "count"
}).rename(columns={"Name": "EmployeeCount"})

# 6. Analysis
print(summary_by_dept)
print(df["SalaryPerAge"].describe())

# 7. Write out results
summary_by_dept.to_csv("dept_salary_summary.csv")
```